Neural Network Projects Report

Alejandro Fernández Trillo

April 22, 2025

Contents

1 Assignment 1: Customer Segmentation Classifier (Fully conected NN) 2
1.1 Problem Description . . . 2
1.2 Tested Architectures and Hyperparameters . . . 2
1.3 Results and Conclusions . . . 8
1.3.1 Final conclussions . . . 10

2 Assignment 2: Neural Network Behavior on Synthetic Dataset 10
2.1 Problem Description . . . 10
2.2 Tested Architectures and hyperparameters . . . 11
2.3 Results and Conclusions . . . 21
2.3.1 Chat GPT help . . . 22

1 Assignment 1: Customer Segmentation Classifier (Fully conected NN)

1.1 Problem Description

In this project, we worked with a dataset provided by an automobile company that aims to expand into new markets using their existing product lines (P1–P5). The company has already implemented a successful marketing strategy in their current market by segmenting customers into four distinct groups (A, B, C, D) and targeting each group with personalized communication.

The main objective of the project was to develop a machine learning model capable of accurately predicting the correct segment for new potential customers based on their demographic and behavioral data. The company provided a dataset with various features such as gender, age, marital status, education level, profession, spending score, and family size. A special column named `Segmentation` represents the label or target variable, which classifies each customer into one of the four segments.

Our task was to build, train, and evaluate a neural network model to classify the customers into these segments and apply the model to a test set of 2,627 new customers. These predictions will help the company replicate their successful marketing strategy in new markets.

This problem represents a real-world business application of machine learning in customer relationship management and market expansion strategies. It also highlights the value of predictive analytics in improving decision-making and targeting customers more effectively.

This dataset is particularly challenging due to the complexity of the features and the intricacies involved in accurately segmenting the customers. Despite multiple attempts from various developers and data scientists, achieving a high level of accuracy on this task has proven to be difficult. The highest accuracy observed by other programmers working with this dataset has been around 49 per cent. This indicates that the model's ability to correctly classify the customers into their respective segments is not straightforward and requires careful tuning and optimization of hyperparameters. The relatively low maximum accuracy achieved so far highlights the complexity of the dataset and the limitations of current models in handling such data.

1.2 Tested Architectures and Hyperparameters

Architectures

- Single-layer neural network (8, 16 neurons)
- Two-layer network [(8,4) (16,8) (32,16) (64,32) (128,64)]
- Three-layer architecture [(16,8,4) (32,16,8) (64,32,16) (124,64,32) (256,128,64)]

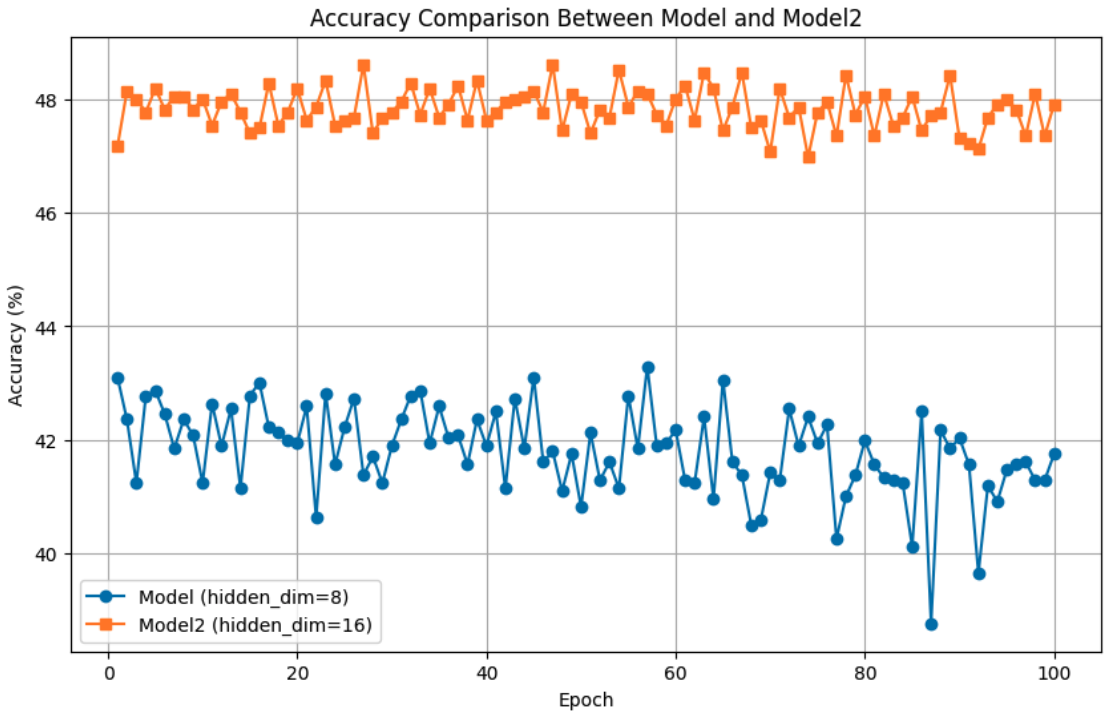


Figure 1: Accuracy of 2 layer NN

	Hidden Layer 1	Hidden Layer 2	Accuracy (%)
0	8	4	48.667602
1	16	8	48.667602
2	32	16	47.592333
3	64	32	46.984572
4	128	64	46.750818

Figure 2: Accuracy of 3 layers

	Hidden Layer 1	Hidden Layer 2	Hidden Layer 3	Accuracy (%)
0	16	8	4	47.498831
1	32	16	8	47.732585
2	64	32	16	46.797569
3	128	64	32	46.937821
4	256	128	64	43.899018

Figure 3: Accuracy of 4 layers

Out of these combinations I,ve found that the most accurate one was the three layers with 16 and 8 neurons in its hidden layers

Hyperparameters

Hyperparameter 1: Learning Rate

The **learning rate** is one of the most important parameters when training a machine learning model. It controls the size of the steps the optimizer takes at each iteration. If the learning rate is too high, the model might miss the best solutions, while if it's too low, the training might become too slow or stall.

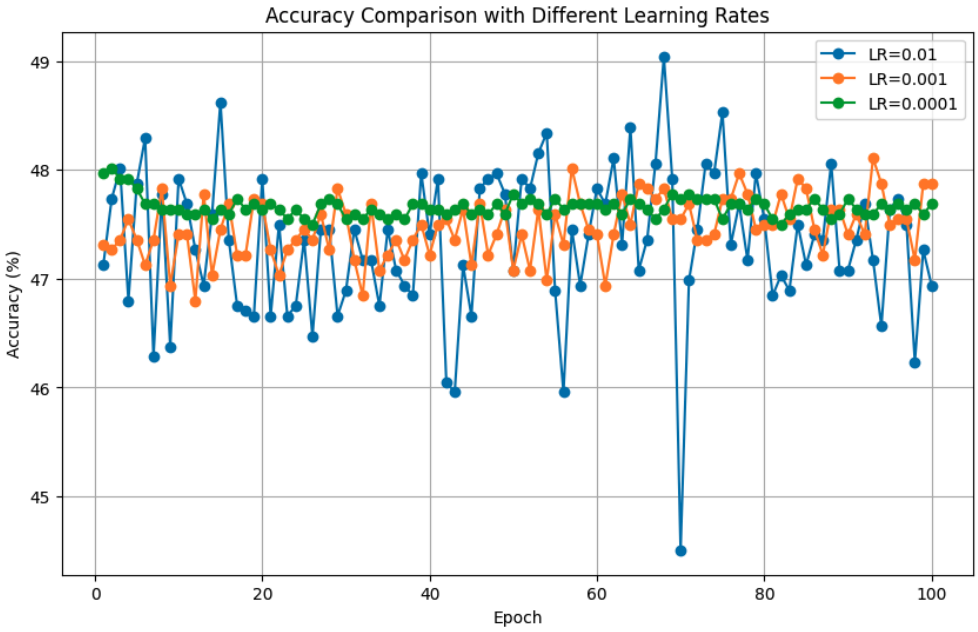


Figure 4: Learning rates accuracy Graph.

Learning Rate	Final Accuracy (%)
0.01	46.9378
0.001	47.8728
0.0001	47.6858

Figure 5: Learning rates accuracy Table.

Results We tested three learning rates: 0.01, 0.001, and 0.0001. In general, a lower learning rate resulted in better accuracy, but also made the training slower. The learning rate of **0.001** performed the best.

A learning rate of 0.001 provides a balance between stability and convergence speed. While 0.01 may cause the model to overshoot minima and diverge, and 0.0001 can lead to very slow learning, 0.001 allows for steady and efficient optimization.

Hyperparameter 2: Batch Size

Batch size is the number of samples processed before the model updates its parameters. A small batch size can lead to faster learning, but might result in higher variability in the accuracy. A larger batch size usually reduces variability but makes the training slower.

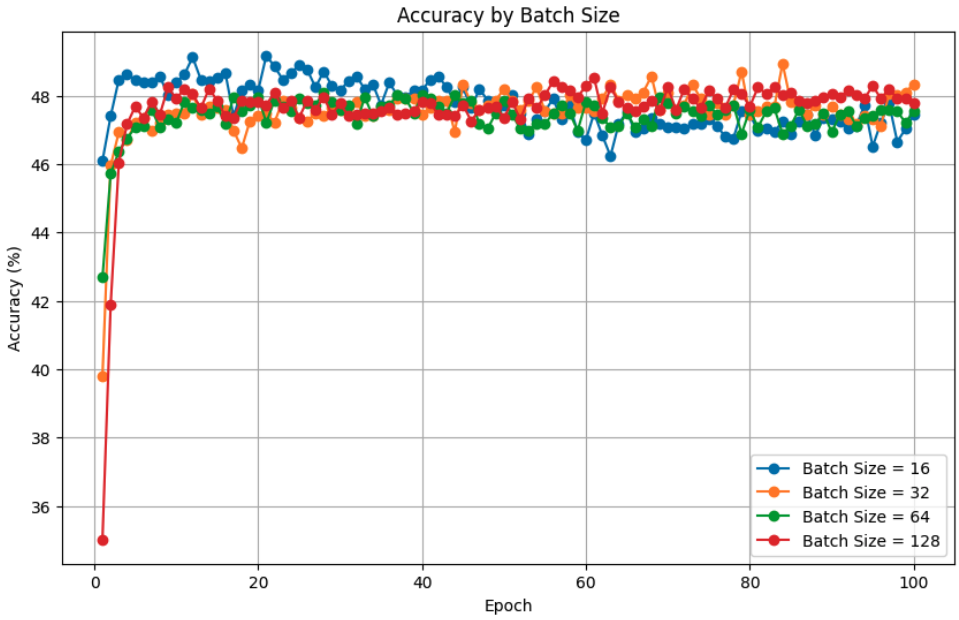


Figure 6: Batch sizes accuracy

Batch Size	Final Accuracy (%)
16	47.4521
32	48.3403
64	47.5456
128	47.7793

Figure 7: Batch sizes accuracy

Results We tested different batch sizes, such as 16, 32, 64, and 128. The results showed that a batch size of **32** provided a good balance between training time and final accuracy.

Hyperparameter 3: Activation Function

Activation functions are crucial because they determine how neurons are "activated" in each layer of the model. We tested **ReLU**, **Sigmoid**, and **Tanh**. ReLU is more common in deep networks because it helps avoid the vanishing gradient problem, while Sigmoid is often useful in simpler models.

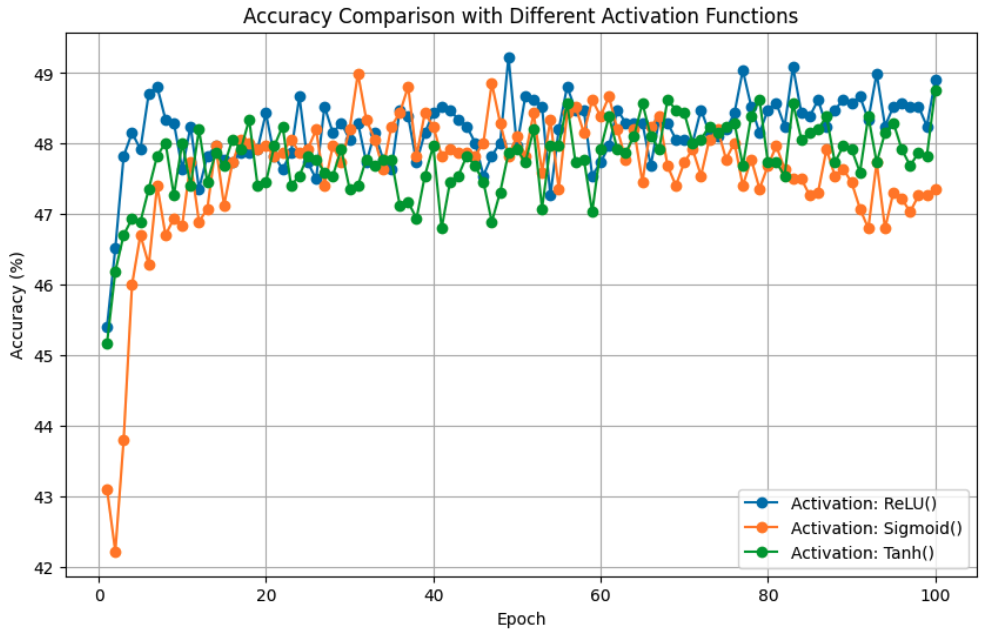


Figure 8: Activation Functions accuracy

Activation Function	Final Accuracy (%)
ReLU()	48.9014
Sigmoid()	47.3586
Tanh()	48.7611

Figure 9: Activation Functions accuracy table

Results The **Relu** activation function worked the best out of all options. ReLU generally performs better than Sigmoid or Tanh in deep networks because it avoids vanishing gradients and allows for faster, more effective training. Its simplicity and non-saturating nature make it ideal for hidden layers.

Hyperparameter 4: Number of Epochs

The **number of epochs** is the number of times the model sees the entire dataset during training. Generally, more epochs allow the model to learn better, but it can also lead to overfitting if the model learns the training data too well or vice versa: underfitting.

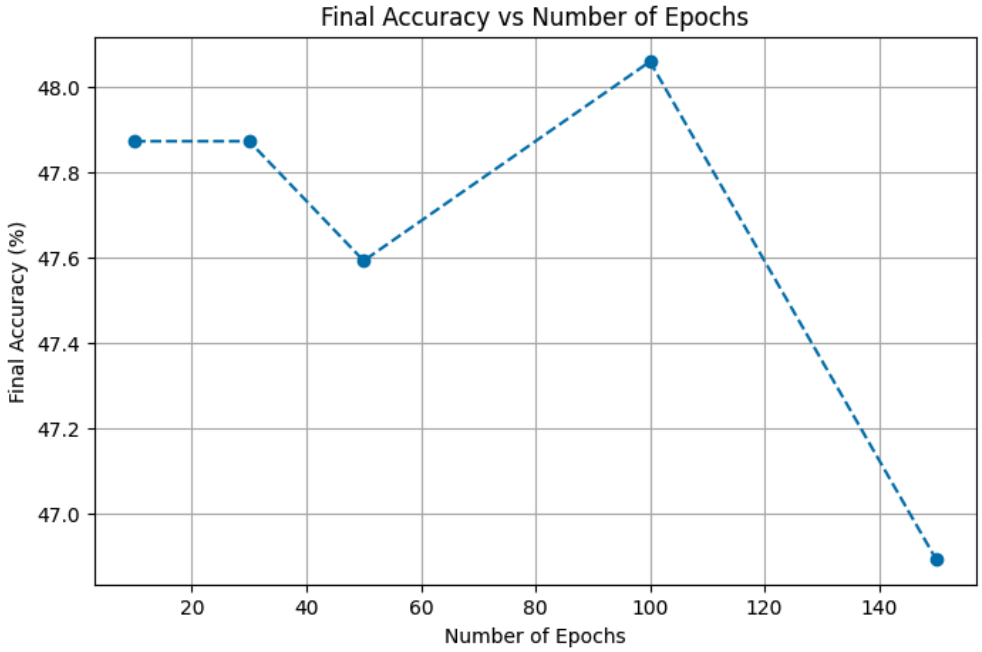


Figure 10: N of epochs accuracy

Epochs	Final Accuracy (%)
10	47.8728
30	47.8728
50	47.5923
100	48.0598
150	46.8911

Figure 11: N of epochs accuracy Table

Results We tested with 10, 30, 50, 100, and 150 epochs. The model with **100 epochs** showed higher final accuracy. Training with 100 epochs provides a good trade-off between underfitting and overfitting. Fewer epochs (10, 30, 50) may not allow the model to learn enough patterns, while 150 epochs can lead to overfitting on the training data.

Hyperparameter 5: Loss Function and Optimizer

We tested two combinations of **loss function** (CrossEntropyLoss and NLLLoss) and **optimizers** (Adam and SGD). The loss function measures how far the model's predictions are from the actual labels. Optimizers like **Adam** and **SGD** update the model's weights based on the gradients calculated.

Loss Function	Optimizer	Accuracy (%)
CrossEntropy	Adam	48.0598
CrossEntropy	SGD	45.3483
NLLLoss	Adam	47.1716
NLLLoss	SGD	46.8443

Figure 12: loss functions and optimizer accuracy

Results The combination of **CrossEntropyLoss** with **Adam** optimizer was the most effective in terms of final accuracy. Adam adapts the learning rate during training, which helps improve convergence. In average, this combination was the one that gave me the most effective results.

As shown in Table, the combination of CrossEntropy loss with the Adam optimizer achieved the highest accuracy. Adam generally provides better convergence due to its adaptive learning rate, especially when used with loss functions that output log-probabilities.

1.3 Results and Conclusions

Best Setup

- Three layers with 16 and 8 neurons
- Relu activation
- Learning rate: 0.001
- Batch size: 32
- Epochs: 100
- optimizer : Adam
- Loss function : CrossEntropy

Performance

- Final accuracy: 48.85%
- Best accuracy during trials: 49.4%

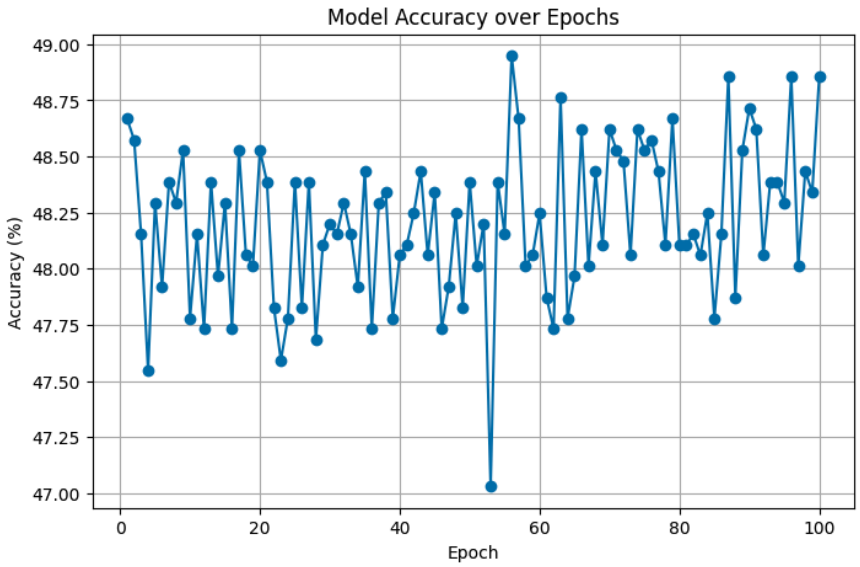


Figure 13: Accuracy graph

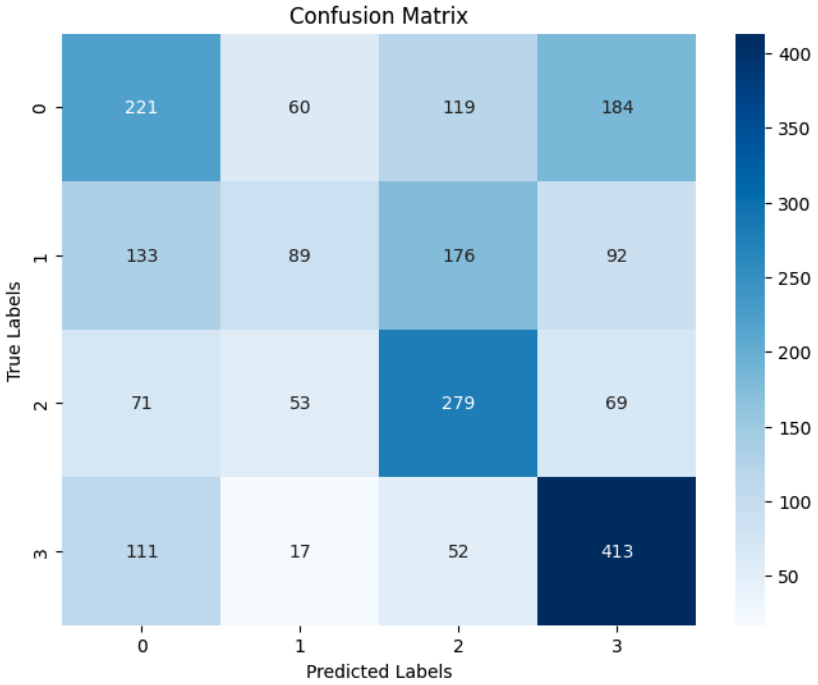


Figure 14: Confusion Matrix

The model struggled especially with Group B customers, who were frequently misclassified as Group C and A. This suggests that the dataset's complexity, not just the model capacity, limits performance.

1.3.1 Final conclussions

Among all the hyperparameters we tested, the combination of **learning rate of 0.001**, **batch size of 32**, **Relu activation function**, and **100 epochs** achieved the best performance for our classification model. However, it's important to note that the **number of layers** and **number of neurons** also play a significant role in the model's performance, and should be adjusted based on the complexity of the data and the specific problem at hand.

This analysis is complex due to the random test and batch it chooses, so in each iteration it will give a **different result**, but these are my conclusions after doing several trials at this large code.

2 Assignment 2: Neural Network Behavior on Synthetic Dataset

2.1 Problem Description

Dataset Description The dataset used in this project contains images representing the letters of the American alphabet (from "A" to "Z") through hand signs performed with a single hand. This dataset is specifically curated for addressing multi-class classification problems and is ideal for applying advanced techniques such as Convolutional Neural Networks (CNN).

- **Classes:** 26 categories corresponding to each letter of the American alphabet.
- **Format:** Images depicting hand signs for each letter.
- **Purpose:** Train a multi-class classification model to accurately identify the sign corresponding to each letter.

Project Objective The main goal is to develop a model based on Convolutional Neural Networks (CNN) capable of correctly classifying the images in the dataset into one of the 26 available categories, achieving an accuracy close to 98% if the algorithm is well-designed and trained.

Potential Applications

- **Accessibility:** Enhancing communication for individuals with hearing disabilities through automatic interpretation of sign language.

- **Education:** Interactive tool for teaching the American alphabet in sign language.
- **Visual Recognition:** Advancements in automatic gesture and sign recognition systems.

2.2 Tested Architectures and hyperparameters

Architectures

- Baseline: 2 convolutional layers
- Extended: 3 convolutionals layers
- Extended: 4 convolutionals layers

We get these results : For the two layers we get a accuracy of **98.9 per cent**

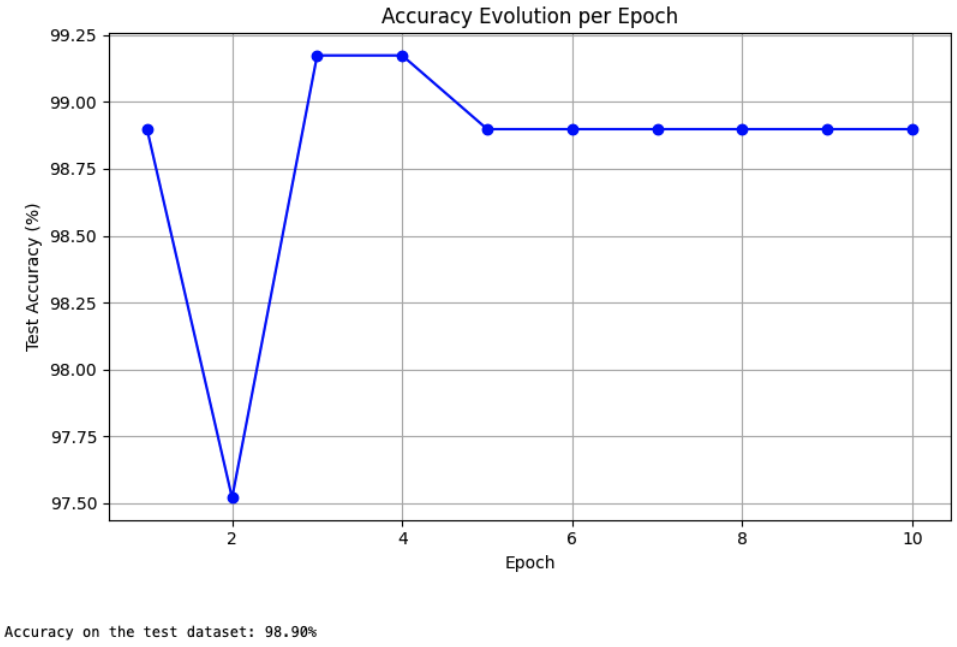


Figure 15: Accuracy graph 3 layers

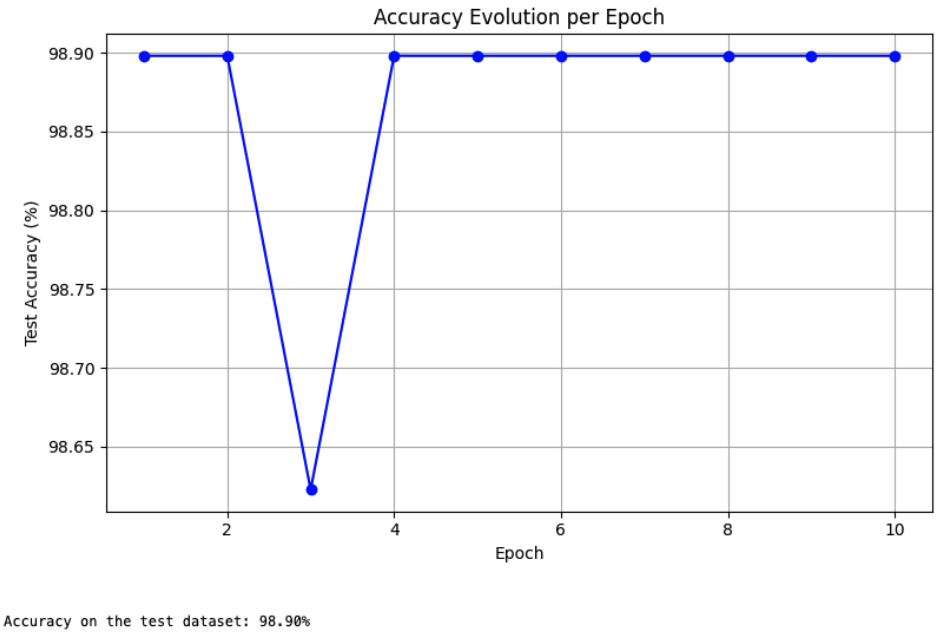


Figure 16: Accuracy graph 4 layers

All three options get the same accuracy so i have decided to choose **3 layers because** :it's a good balance between simplicity and learning capacity. Two layers might not capture enough complex features, while four could add unnecessary complexity and risk overfitting. With 3 layers, the model remains efficient while still being deep enough to learn rich and useful patterns from the data.

Hyperparameters

Note: The first 3 hyperparameters : Learning rate , Batch size and N of epochs were tested with the architecture of 2 layers.

Because in the code the architectures were the fourth hyperparameter tested and from there we continued with the best architecture performing layers.

Hyperparameter 1 : Learning rate

The **learning rate** is one of the most important parameters when training a machine learning model. It controls the size of the steps the optimizer takes at each iteration. If the learning rate is too high, the model might miss the best solutions, while if it's too low, the training might become too slow or stall.

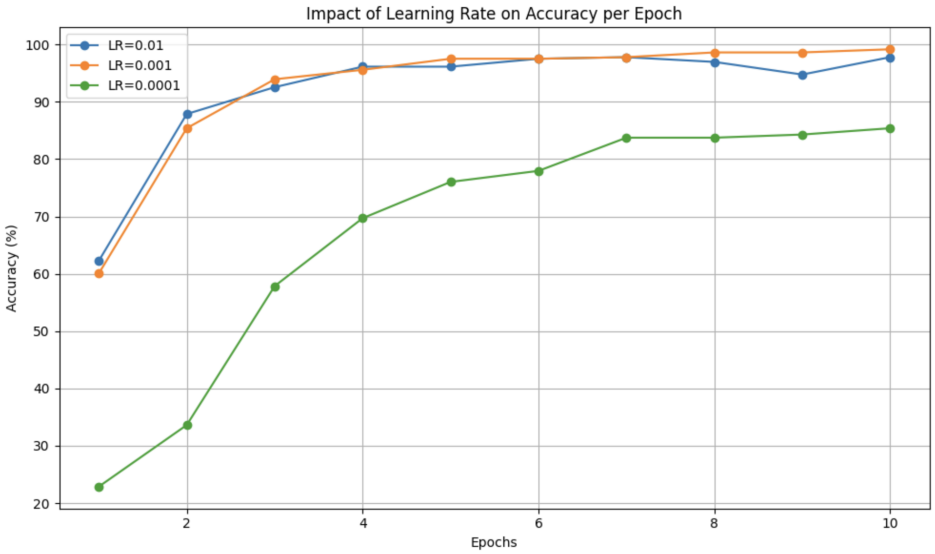


Figure 17: Accuracy graph Learning rates

Summary Table:

Learning Rate	Final Accuracy (%)
0.01	97.7961
0.001	99.1736
0.0001	85.3994

Figure 18: Table accuracy

Results We tested three learning rates: 0.01, 0.001, and 0.0001. The learning rate of **0.001** performed the best.

A learning rate of 0.001 provides a balance between stability and convergence speed. While 0.01 may cause the model to overshoot minima and diverge, and 0.0001 can lead to very slow learning, 0.001 allows for steady and efficient optimization.

Hyperparameter 2: batch size

Batch size is the number of samples processed before the model updates its parameters. A small batch size can lead to faster learning, but might result in higher variability in the accuracy. A larger batch size usually reduces variability but makes the training slower.

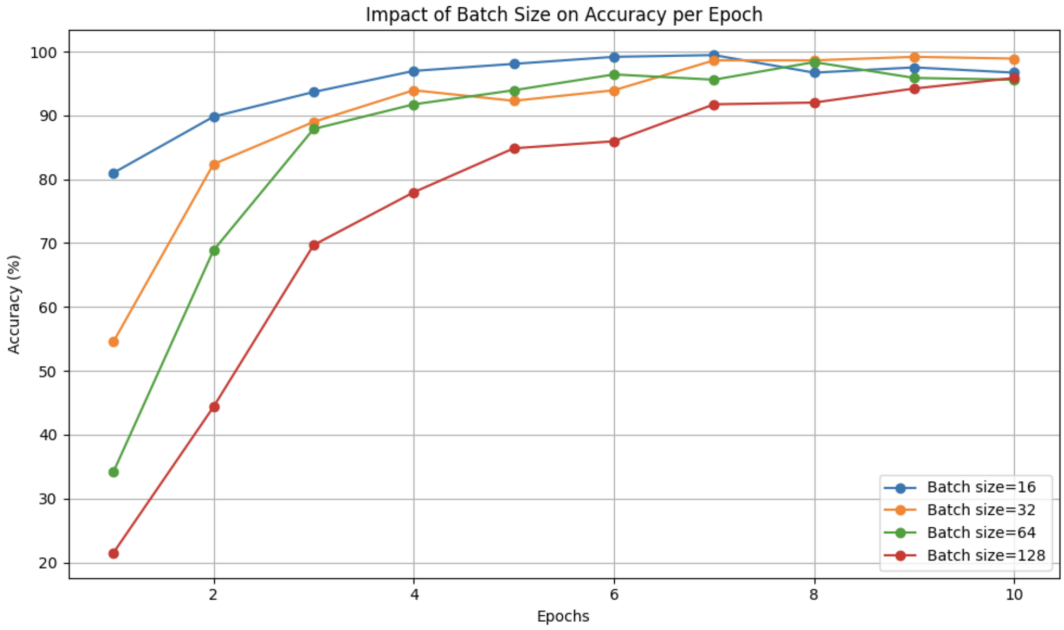


Figure 19: Accuracy graph Batch sizes

📊 Summary Table:

Batch Size	Final Accuracy (%)
16	96.6942
32	98.8981
64	95.5923
128	95.8678

Figure 20: Table accuracy

Results We tested different batch sizes, such as 16, 32, 64 and 128. The results showed that a batch size of **32** provided a good balance between training time and final accuracy

Hyperparameter 3: N of epochs

The **number of epochs** is the number of times the model sees the entire dataset during training. Generally, more epochs allow the model to learn better, but it can also lead to overfitting if the model learns the training data too well or vice versa: underfitting.

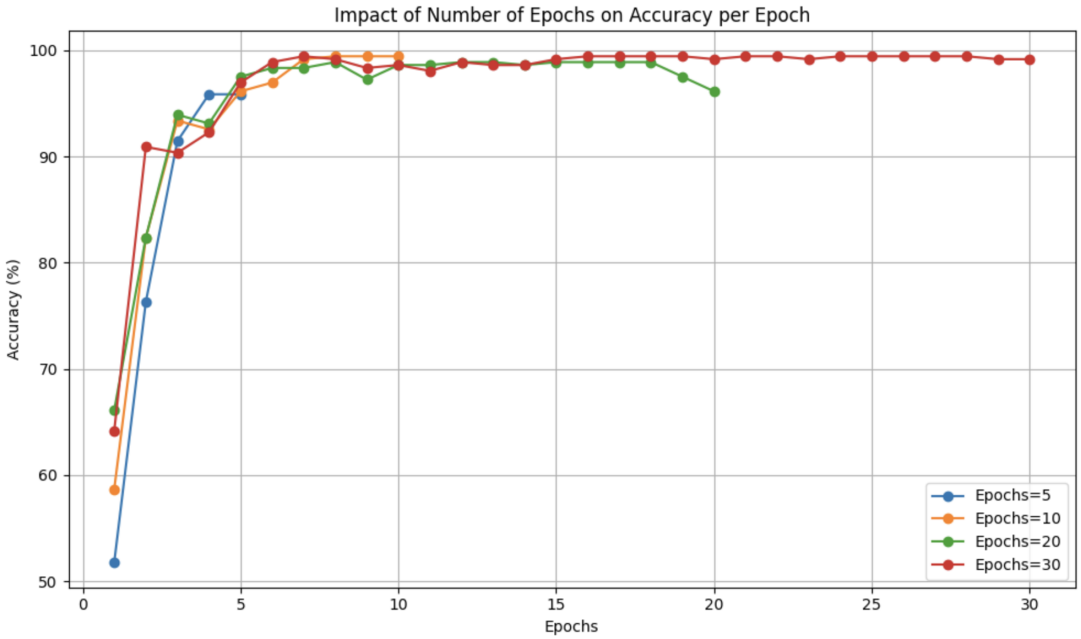


Figure 21: Accuracy graph N of epochs

c

Results We tried these values : 5 ,10, 20, 30. The best accuracy in your CNN appears at 10 epochs because it's the most reasonable balance between underfitting and overfitting. With 5 epochs, the model may not have learned enough patterns from the data yet. With 20 or 30 epochs, the model might start overfitting—memorizing the training data instead of generalizing well to unseen data. Therefore, **10** epochs offer a sweet spot where the model learns enough without starting to overfit.

Hyperparameter 4:N of kernels

The kernel size affects how much of the image the model "looks at" when extracting features. Smaller kernels like 3x3 focus on fine details and allow the network to go deeper while maintaining control over the number of parameters.

We try these four combinations : (3-3-3),(5-3-3),(5-5-3) and (7-5-3)

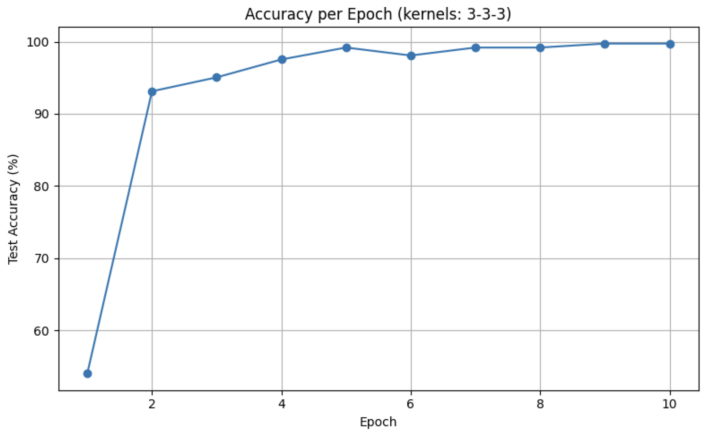


Figure 22: (3-3-3) Kerenels accuracy

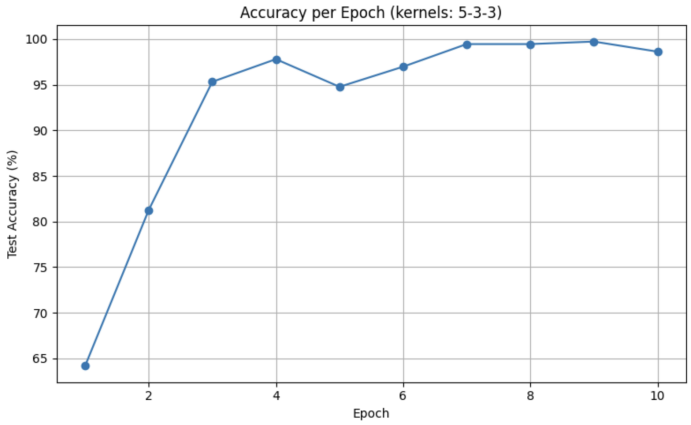


Figure 23: (5-3-3) Kernel accuracy

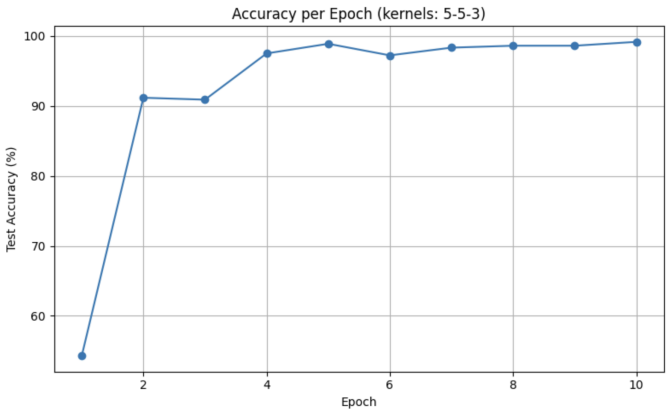


Figure 24: (5-5-3) Kernel accuracy

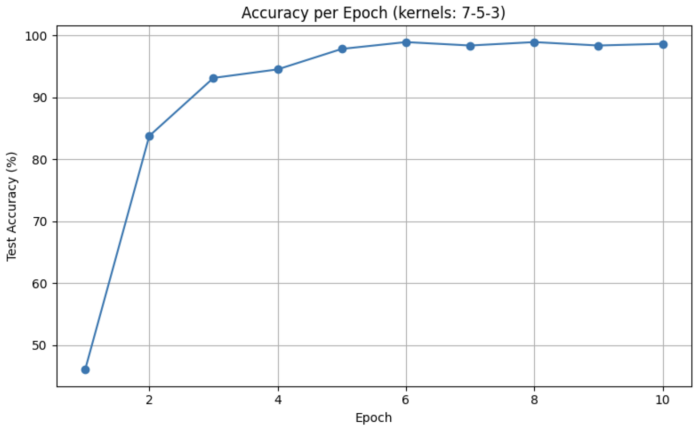


Figure 25: (7-5-3)Kernel accuracy

Kernel Configuration	**Accuracy (%)**
(3, 3, 3)	99.72
(5, 3, 3)	98.62
(5, 5, 3)	99.47
(7, 5, 3)	98.62

Table 1: Accuracy obtained with different kernel configurations

Results The best performance came from **(3, 3, 3)** because small kernels help the model learn detailed patterns step by step, layer by layer. They also reduce the risk of overfitting and make the model more efficient. This setup is often effective because stacking multiple 3x3 convolutions can capture complex features while keeping the architecture simple and powerful.

Hyperparameter 5:N of filters

The number of filters determines how many features the model can extract at each layer. More filters allow the CNN to learn more complex and detailed patterns from the images.

filter configs = (32, 64, 128), (64, 128, 256), (128, 256, 512)

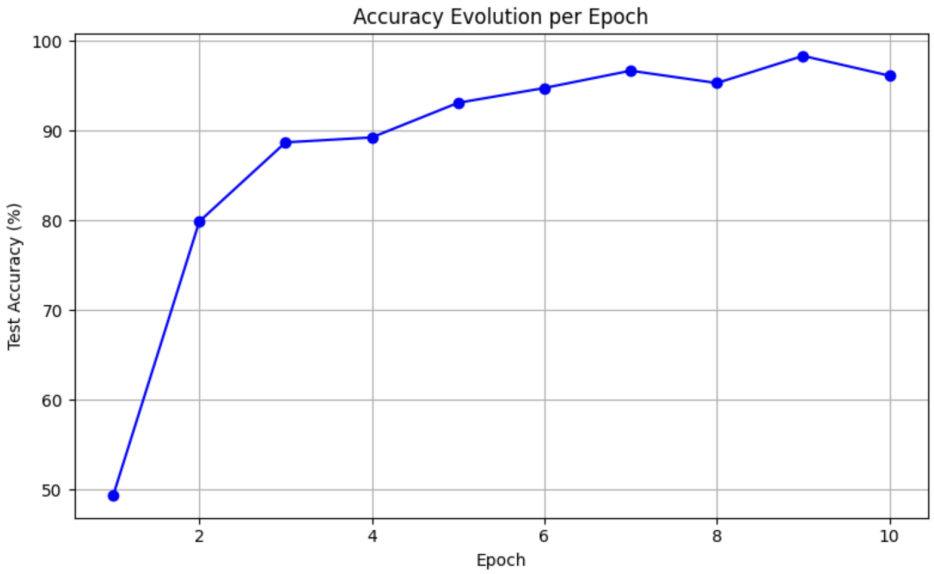


Figure 26: Accuracy Filters (32,64,128)

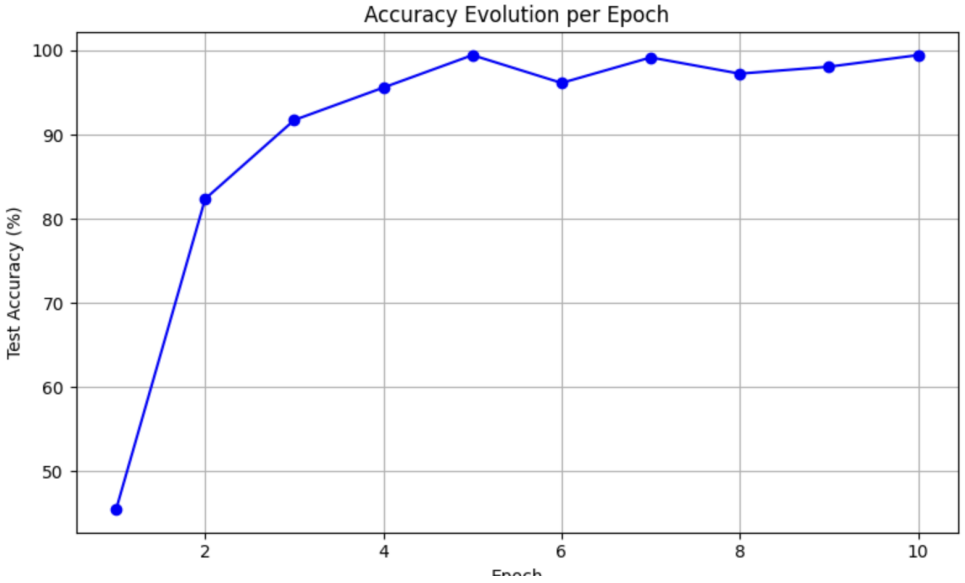


Figure 27: Accuracy Filters (64,128,256)

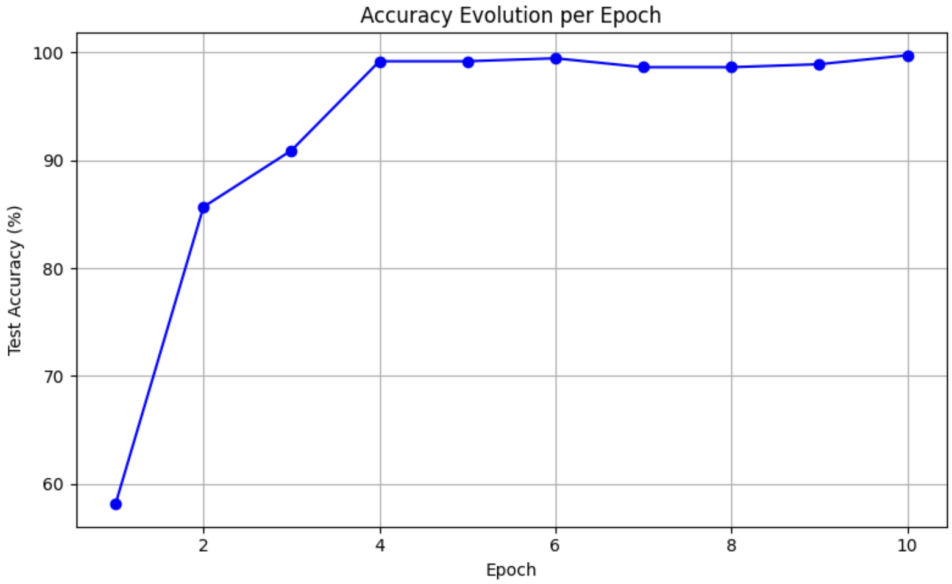


Figure 28: Accuracy Filters (128,256,512)

N of filters	Accuracy (%)
(32, 64, 128)	96.14
(64, 128, 256)	99.45
(128, 256, 512)	99.72

Table 2: Accuracy obtained with different kernel configurations

Results The best performance came from **(128, 256, 512)** because this combination gives the model a higher capacity to detect fine-grained features and deeper structures in the images. While it increases computational cost, it leads to better accuracy since the model can learn more expressive and powerful representations of the data.

Padding ans strike

I have decided to not change this values because

I decided not to change these values because using **stride = 1 and padding = 1** helps the model keep the size of the images almost the same after each convolution. This way, the model can learn more details without making the images too small too quickly. If I increased the stride or changed the padding, the model might miss important parts of the image, which could reduce accuracy.

Activation functions

Activation functions are crucial because they determine how neurons are "activated" in each layer of the model. We tested **ReLU**, **LikelyRelu**,**Sigmoid**, and **Tanh**. ReLU is

more common in deep networks because it helps avoid the vanishing gradient problem, while Sigmoid is often useful in simpler models.

Activation Function	Final Test Accuracy
ReLU	98.62%
LeakyReLU	99.45%
Sigmoid	3.31%
Tanh	99.17%

Figure 29: Activation functions Accuracy

Result I have tried these examples : Relu , LikelyRelu , Tanh , sigmoid. The best one was the relu in previous examples, so we chose the relu over the likely relu that was the best in performance.

Relu : 99.72 ¿ Likely Relu 99.45

Loss functiosn and optimizers

We tested two combinations of **loss function** (CrossEntropyLoss and NLLLoss) and **optimizers** (Adam and SGD). The loss function measures how far the model's predictions are from the actual labels. Optimizers like **Adam** and **SGD** update the model's weights based on the gradients calculated.

Optimizer	Loss Function	Final Test Accuracy
Adam	CrossEntropyLoss	99.72%
Adam	NLLLoss	99.72%
SGD	CrossEntropyLoss	4.68%
SGD	NLLLoss	5.51%

Figure 30: Loss functions/Optimizers accuracy

Results The combination of **CrossEntropyLoss** with **Adam** optimizer was the most effective in terms of final accuracy. Adam adapts the learning rate during training, which helps improve convergence. With this combination we ot previusly 99.72

2.3 Results and Conclusions

Best Setup

- Three-layer architecture
- 3-3-3 kernels
- 128-256-512 filters
- ReLU activation
- Learning rate: 0.001
- Batch size: 32
- Epochs: 10
- Crossentropyloss and ADam

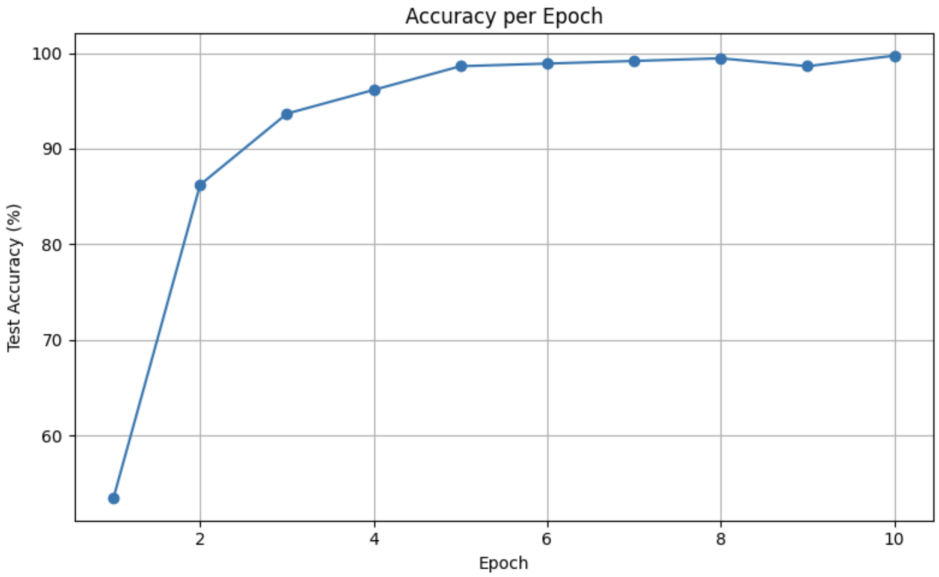


Figure 31: Final model

Final accuracy of 99.72 per cent

Performance

- Accuracy: close to 100% on training and test set
- Fast convergence and no overfitting detected

The experiment confirmed that simpler models are sufficient for linearly separable data. More complex architectures led to slight overfitting without significant gains in performance.

2.3.1 Chat GPT help

I have used the help of this ai with two basic process:

- With all the hyperparameters tuning at the time of doing the loops for each hyperparametes and putting all the data into an array and then plot it i have done that with the help of this , but i can assure that i have learned how it does
- And the tabulate idea of representing more clearly the information was idea of chat gpt ass well
- And all the plot as well beacuse im very bad at using the matplot.lib

General Observations

Across both assignments, the importance of tuning learning rate and architecture depth became clear. Additionally, batch size and activation function choices greatly influenced training stability and final performance.